Engineering Science and Technology, an International Journal 26 (2022) 100995

HOSTED BY

Contents lists available at ScienceDirect

Engineering Science and Technology, an International Journal

journal homepage: www.elsevier.com/locate/jestch

A multi-objective approach for energy-efficient and reliable dynamic VM consolidation in cloud data centers

Monireh H. Sayadnavard [a], Abolfazl Toroghi Haghighat [b,*], Amir Masoud Rahmani [c]

[a] Department of Computer Engineering, Bandar Anzali Branch, Islamic Azad University, Bandar Anzali, Iran
[b] Faculty of Computer and Information Technology Engineering, Qazvin Branch, Islamic Azad University, Qazvin, Iran
[c] Future Technology Research Center, National Yunlin University of Science and Technology, Yunlin, Taiwan

ARTICLE INFO

Article history:
Received 16 July 2020
Revised 13 April 2021
Accepted 26 April 2021
Available online 15 May 2021

Keywords:
Cloud computing
VM consolidation
Reliability
Markov chain
Multi-objective optimization
ε-MOABC algorithm

ABSTRACT

The rapid growth of cloud computing in the last decade has led to an increasing concern about the energy requirement of cloud data centers. Dynamic virtual machine (VM) consolidation is an effective way to tackle this issue, where VMs are executed on as few physical machines (PMs) as possible. Meanwhile, VM placement must be performed strategically, by considering different factors of the available resources to optimal exploitation of them. Moreover, a major challenge is the system's reliability degradation because of the high frequency of consolidation and placing VMs on unreliable PMs. In this paper, we address the problem by introducing a discrete-time Markov chain (DTMC) model to predict future resource usage. Using the DTMC model along with the reliability model of PMs leads to more accurate PMs categorization based on their status. Then, a multi-objective VM placement approach is proposed to achieve the optimal VMs to PMs mapping using the ε-dominance-based multi-objective artificial bee colony (ε-MOABC) algorithm which can efficiently balance the overall energy consumption, resource wastage, and the system reliability to meet SLA and QoS requirements. We have validated the effectiveness of our proposed approach by conducting a performance evaluation study using the CloudSim toolkit. Competitive analysis of the experimental results demonstrates that the proposed approach significantly improves energy consumption while avoiding the inefficient VM migrations.

1. Introduction

The emerging cloud computing is a model for enabling ubiquitous, convenient, and on-demand access to a shared pool of customizable computing resources (e.g. servers, storage, networks, applications, and services) that can be provisioned with minimal service provider interactions and management efforts [1]. Cloud data centers consume enormous amounts of energy resulting in high operating costs and carbon emissions. The reason for high energy consumption is not just the amount of computing resources but rather lies in the inefficient use of these resources and huge energy wastage. Furthermore, to satisfy customer's expectations concerning performance, achieving the desired level of Quality of Service (QoS) between cloud service providers and their users is a critical issue. The QoS requirements are characterized by Service Level Agreements (SLAs) that define the required performance levels, such as system throughput, response time and downtime ratio. As a consequence, the main objective is to reduce energy consumption while preserving QoS requirements. Toward this goal, an effective way to improve resource utilization and energy efficiency is VM consolidation that has been widely studied in recent years [2,3]. Evidently, one of the most important factors in energy waste is the existence of many idle PMs. Thus, the main idea of the VM consolidation is to execute the VMs on as few PMs as possible using hardware virtualization technology [4] and VM live migration to reduce the energy consumption and the number of PMs powered on [5]. There are several challenges on the VM consolidation problem that still need further research. Many methods of VM consolidation has focused on the energy and system performance trade-off. Indeed, decreasing energy consumption may lead to degrade the performance and increase the SLA violations due to the growing of the number of live migrations. Another challenge that needs to be addressed is the negative effect of the high frequency of VM consolidation on the system reliability [6–9]. VM consolidation may increase the server failure probability and compromise the system reliability by increasing the load on some servers. Moreover, switching the power state of a server from idle to low-power state and vice versa reduces the server reliability in

* Corresponding author.
E-mail address: haghighat@qiau.ac.ir (A. Toroghi Haghighat).
Peer review under responsibility of Karabuk University.

https://doi.org/10.1016/j.jestch.2021.04.014
2215-0986/

addition to wasting energy [10]. Therefore, an efficient dynamic VM consolidation approach must take into account distinct factors while meeting SLA and QoS requirements.

In this paper, we present an effective dynamic VM consolidation approach by developing different methods in each phase of consolidation. First, we extend our previous work [11] to improve PMs categorization using the short-term prediction of future utilization based on a Markov chain model. Second, using the obtained results from the previous step and the mean first passage time concept of the Markov chain, a VM selection policy has been introduced to reduce the number of migrations. In contrast to existing approaches, VMs on underloaded PMs will be added to the migration list at the end of the VM selection phase instead of exploring such PMs in the final step. The reason is performing VM placement more efficient with a comprehensive view of the whole possible allocations to optimal usage of available resources. Then, we address the VM placement problem as a multi-objective optimization problem to achieve an equilibrium between energy consumption, resource wastage, and system reliability. Considering resource wastage leads to optimal usage of available resources and as a result, a PM can host more VMs. Furthermore, as mentioned above reliability is a challenging factor that the majority of existing methods do not take it into account. It is important to consider the reliability of PMs in the consolidation process to prevent extra migrations and reduce energy wastage. In such a complex multi-objective problem, traditional optimization methods are not able to efficiently provide proper results and deal with the difficulties of exploring solution spaces. Hence, novel approaches based on bio-inspired meta-heuristic schemes, seem to be the most promising options to achieve a better trade-off between different objectives. In this direction, we adopted the ε-MOABC [12] algorithm to model the VM placement and dynamics of a typical cloud environment, with the aims of minimizing the total energy consumption, resource wastage, and maximizing the system reliability. Although our algorithm does not directly consider QoS as an optimization objective, but reducing the PMs overloading risk, as well as the number of migrations result in effectively guaranteeing QoS. Specifically, our major contributions can be summarized as follows:

- Proposing a new model for PMs categorization based on two distinct Discrete Time Markov Chain (DTMC) and Continuous Time Markov Chain (CTMC) to determine the system behavior in terms of CPU utilization and reliability of the PMs. The idea underlying our solution is to improve the accuracy of PMs categorization and consequently reducing the number of migrations. Unlike existing works that have used a variety of Markov chain models only for forecasting, we apply the mean first passage time concept, derived from the Markov model, during the VM selection phase to avoid unnecessary migrations.
- Introducing a novel heuristic VM selection policy considering the migration time, as well as the number of migrations to reduce the negative impact of migrations on QoS. A particular strength of this policy is investigating the task completion time while selecting VMs for migration. In this regard, a constraint is defined to select only those VMs that cannot complete their tasks before mean first passage time in the current PM to prevent extra VM migration that leads to improving QoS.
- Presenting a multi-objective VM placement method based on the ε-MOABC algorithm which can successfully balance the overall energy consumption, resource wastage, and the system reliability to meet SLA and QoS requirements. The main idea of integrating these three objectives is a balanced and reliable VM allocation, resulting in a reduction of additional migrations in the long term. Furthermore, to the best of our knowledge, this is the first work that applies MOABC to address the VM placement problem.
- Experimental evaluation to validate the effectiveness of our proposed approach utilizing the CloudSim as a simulation framework.

The rest of the paper is organized as follows: The current and past research on VM consolidation are discussed in Section 2. The system architecture and problem statements are explained in Section 3. Our proposed approach for dynamic VM consolidation including DTMC based prediction model, VM selection algorithm, and ε-MOABC based VM placement are described in detail in Section 4. The experimental setup and results are illustrated in Section 5. Finally, conclusions and future works are presented in the last section.

2. Related work

In this section, the relevant research works in the literature related to the VM consolidation problem are discussed.

Many existing studies have been formulated the VM consolidation as a well-known NP-hard bin packing problem [13–17]. To solve this NP-hard problem, several heuristics like greedy algorithms are utilized to approximate the optimal solution. Two popular algorithms of them are First Fit Decreasing (FFD) and Best Fit Decreasing (BFD) [17]. The authors in [14] have divided VMs consolidation into several sub-problems due to the complexity of the problem, and then they have proposed the novel adaptive heuristics for each sub-problem. They have used the Modified Best Fit Decreasing (MBFD) algorithm to solve the VM placement problem by considering power consumption and SLA violations. This algorithm allocates VMs to the hosts in such a way that having a minimum increase of energy consumption. They have also conducted several methods for VM selection in [18], including Minimum Migration Time (MMT), Random Selection (RS), and Maximum Correlation (MC). Then, they have proposed the Power-Aware Best Fit Decreasing (PABFD) algorithm to determine the destination host.

The authors in [19] have proposed a burstiness-aware server consolidation algorithm and named it QUEUE. First, the burstiness of workload is captured using a two-state Markov chain, then to avoid live migrations, some extra resources on each PM is reserved. Shen et al. in [20] proposed a mechanism that predicts the VM resource utilization patterns and consolidates complementary VMs with spatial/temporal awareness into one PM to reduce the number of PMs, maximize resource utilization and reduce the number of VM migrations. Beloglazov and Buyya [21] investigated the host overload detection problem by maximizing the mean time between VM migrations under a specified QoS goal for any known stationary workload using a Markov chain model. The unknown nonstationary workloads are also handled using the multi size sliding window approach. In [22] a VM placement algorithm is proposed that considers resources, virtual machine status, QoS metrics, and I/O data using priority based probability queuing model. During the VM placement phase, data location is considered to avoid unnecessary migration. The authors in [23] studied the influence of the dynamic workload, CPU utilization, times of VM migrations, opportunity of VM migration, energy consumption and QoS from nine related factors. They developed a Bayesian Network-based estimation model (BNEM) that each node represents one aspect of VM migration. In [24] a prediction-based VM consolidation approach is presented that considers both an estimation of future requested resources using Kernel Density Estimation technique, and future migration traffic to reduce the number of migrations. Sharma et al. [9] proposed a failure-aware and

energy-efficient VM consolidation mechanism which takes the occurrence of failures and the hazard rate of physical resources into account to reduce the energy consumption without compromising the system reliability. They also proposed a failure prediction technique based on the mathematical models to trigger the VM migration and VM checkpointing mechanisms.

Various methods have been proposed to solve the VM consolidation problem as an optimization problem using evolutionary algorithms. Ant Colony Optimization (ACO) algorithm has been used in many VM consolidation methods to optimize different objectives [25–28]. In [26], the ACO algorithm is used to place the VMs into the least number of PMs while satisfying QoS requirements. To find a near optimal solution a multi-objective function is defined that considers the number of sleeping PMs and the number of migrations. Zhao et al. [28], introduced a nonlinear model to quantify PM power consumption and VM performance models to show performance degradation caused by resource sharing among VMs. Then, VM placement is formulated as an NP-hard bi-objective optimization problem, which is solved using an ACO based algorithm. In [29], a Genetic Algorithm (GA) based method called GABA is presented that dynamically finds the optimum reconfiguration for a set of VMs according to the predicted future demand of the workload. In [30], a VM consolidation approach is proposed which searches the optimal mapping between VMs and PMs to minimize energy consumption and the number of migrations using Artificial Bee Colony (ABC). In this approach, a multi-objective optimization model is established and then multiple objectives converts into a single objective problem. Li et al. [31] developed an improved discrete differential evolution algorithm to find optimal solution for the VM placement problem. They tried to decrease energy consumption and overloading risk, as well as improving QoS.

As mentioned above, the main drawback of the most existing studies is that they focused only on reducing the number of active PMs using VM live migration to prevent inefficient resource usage. Although these methods are very efficient in energy management, but they ignore the negative impact of high frequency of VM consolidation on the system reliability and the desired QoS requirements cannot be achieved. Therefore, a holistic view and various factors evaluation in the decision process is essential. Our study considers different targets including energy consumption, resource wastage, and the system reliability simultaneously which leads to notable improvements in output results.

3. System architecture

We consider a system that consists of a single data center composed of N heterogeneous PMs and various resource capacities. Each PM is characterized by its CPU performance defined in millions of instructions per second (MIPS), amount of RAM, network bandwidth and disk storage. Let PM = {PM_1, PM_2, ..., PM_i,..., PM_m} be the set of active PMs in the current state of the data center and VM = {VM_1, VM_2,..., VM_j,..., VM_n}be the set of VMs deployed on a PM that are managed by Virtual Machine Monitor (VMM). In our proposed approach, the VMs are initially allocated to a random PM in the data center based on its resource requirements. At any given time, users submit their requests for provisioning *n* heterogeneous VMs, which are characterized by requirements of resources. Each request has different tasks and resource requirements. It is assumed that submitted tasks are independent. Failing a running VM will cause the running tasks on it to fail, but will not affect the entire request. Each task also contains a number of subtasks that can use different VM resources. We have assumed that tasks scheduled on a single VM will execute in sequential order.

The system architecture is considered a two-level hierarchical model consists of two kinds of agents: fully distributed Local Agents (LAs) in PMs, and a Global Agent (GA) resides in a master node [26]. Local agents are responsible for identifying the status of PMs. The global agent also receives the status of each PM from the relevant local agent and decides how to implement the consolidation process and sends the instructions to the VMMs based on the migration plan obtained by solving a multi-objective optimization problem. In the proposed approach, each local agent has two main functions: predicting server reliability and predicting CPU utilization. The problem is to minimize the number of PMs used, by maximizing the resource utilization in each PM using live migration of VMs so that the freed PMs can be set to a power saving state. The system architecture is depicted in Fig. 1.

4. Proposed VM consolidation approach

The main purpose of the presented approach is to reduce unnecessary migrations by predicting the status of PMs and selecting appropriate migration plan. To this end, it has been attempted to allocate VMs on safer PMs by determining the probabilistic status of PMs. In the first phase of our proposed approach, PMs status are predicted based on the historical data in two different areas. Each local agent estimates PMs utilization and reliability using two distinct Markov chain models. The second phase identifies the VMs that need to migrate. The third phase determines the appropriate target PMs based on multi-objective optimization. Finally, by shutting down the additional idle PMs in the fourth phase, the VM consolidation process is completed. It should be noted that a PM in the sleep mode is switched on only when it is not possible to migrate a VM to an already active PM. At the beginning of each round of the VM consolidation during the initial VM deployment, if we need to turn on a PM, it is selected based on the resource requirements and available resources. Fig. 2 shows the main steps of the proposed VM consolidation method.

4.1. PMs categorization

This step of VM consolidation process is performed by local agents distributed on the PMs. Each local agent determines PM status using historical data and two distinct Markov chains. In this regard, a DTMC model as an efficient tool for analysis of complex systems in uncertain environments is used to forecast the load status of the PMs. Furthermore, a CTMC model is used to predict PMs reliability. The CTMC model has been explained in detail in our previous work [11]. Formal definitions of DTMC and CTMC are as follows [32]:

Definition 1. *A stochastic process* $\{X_n,\ n \geq 0\}$ *on state space S is said to be a discrete-time Markov chain (DTMC) if, for all i and j in S*

$$P(X_{n+1} = j | X_n = i, X_{n-1}, \cdots, X_0) = P(X_{n+1} = j | X_n = i) \quad (1)$$

This equation implies that the conditional probability on the left-hand side is the same no matter what values $X_0, X_1, \cdots, X_{n-1}$take. In the other word, given the present state of the system (X_n), the future state of the DTMC (X_{n+1}) is independent of its past ($X_0, X_1, \cdots, X_{n-1}$).

Definition 2. *A stochastic process* $\{X(t),\ t \geq 0\}$ *on state space S is called a continuous-time Markov chain (CTMC) if, for all i and j in S and t,* $s \geq 0$,

$$P(X(s+t) = j | X(s) = i, X(u), 0 \leqslant u \leqslant s) = P(X(s+t) = j | X(s) = i) \quad (2)$$

This equation implies that the evolution of a CTMC from a fixed time s onward depends on $X(u)$ (the history of the CTMC up to time s) only via $X(s)$ (the state of the CTMC at time s).

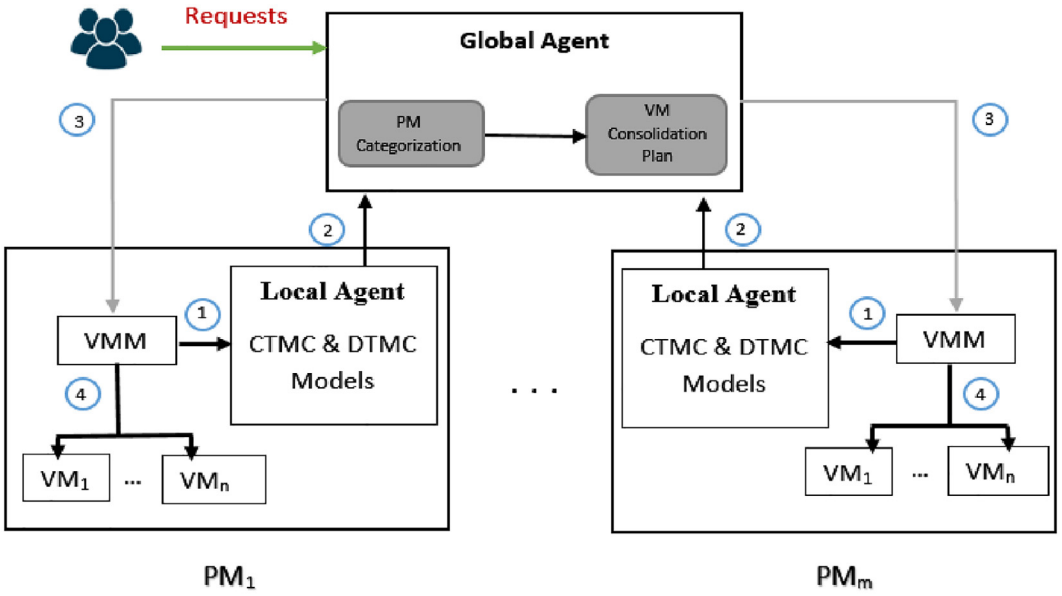

Fig. 1. System architecture.

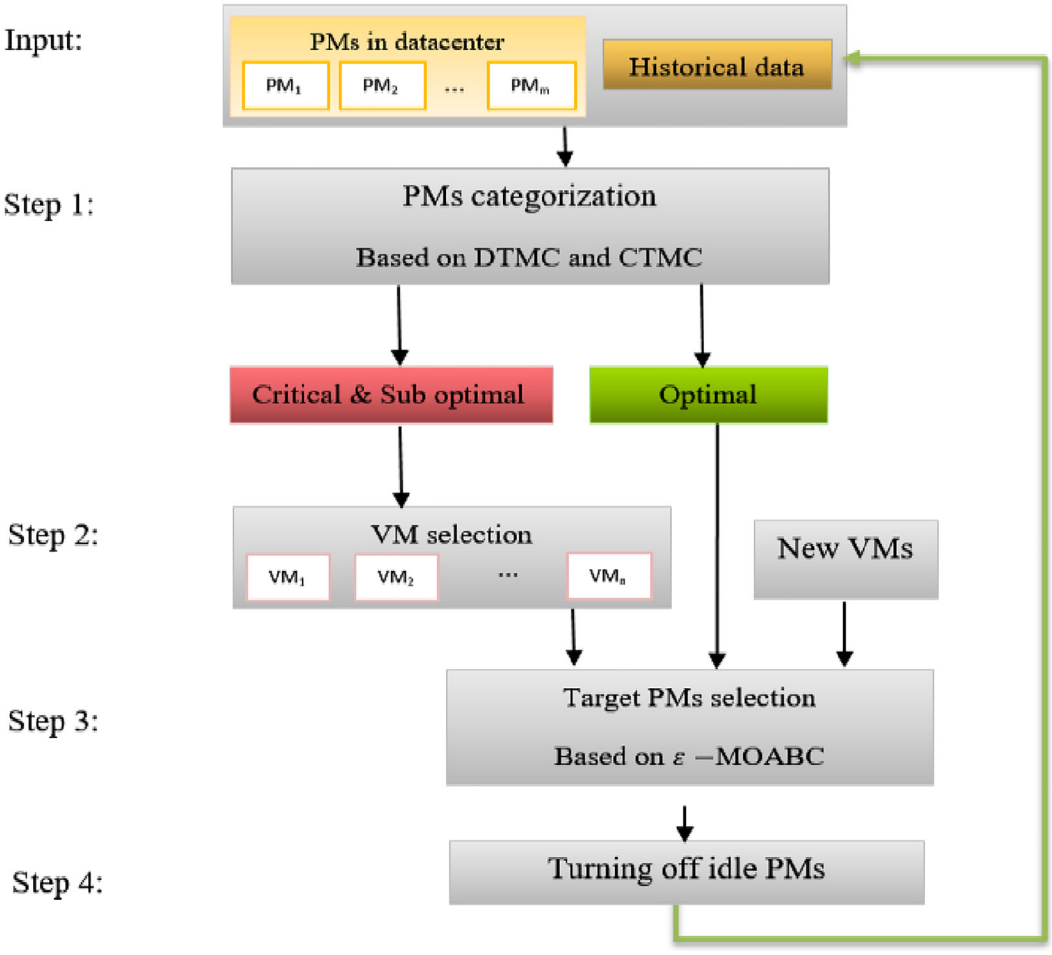

Fig. 2. Proposed VM consolidation steps.

Definition 3. *The expected value of random time at which a stochastic process first passes into a given subset of the state space is called first passage time* [32]. *These are sometimes referred to as mean first passage time*(*MFPT*). *Let* $\{X(n), n \geq 0\}$ *be a DTMC on state space* $S=\{1, 2, 3, \ldots, N\}$ *with transition probability matrix P. The first passage time into state N defined as*

$$T = \min\{n \geq 0 : X_n = N\} \tag{3}$$

Where T is the (random) number of steps that it takes to actually visit state N. Typically T can take values in {0, 1, 2, 3, ...}. The expected value of this random number E(T) is defined as

$$m_i = E((T|X_0 = i)) \tag{4}$$

According to a theorem defined in [32], the expected first passage time satisfy the following relation,

$$m_i = 1 + \sum_{j=1}^{N-1} p_{ij} m_j, \quad 1 \leq i \leq N-1 \tag{5}$$

Where p_{ij} is the entry of the transition probability matrix P.

4.1.1. DTMC-Based host load status prediction

The set of VMs assigned to a PM over a period of time determines the PM workload. There are many factors such as CPU utilization, memory usage and storage, that are effective in determining PM workloads. But often the most effective factor is the CPU utilization [14]. Here, we have assumed that the workload is stationary and known [21]. In this way, the data collected on the CPU utilization of the hosts at regular time intervals can be modelled by a discrete time Markov chain. Indeed, the state transition probability is not dependent on the current moment, and therefore it can be described as a time homogeneous DTMC. It is also assumed that an underload PM does not directly switch to overload state. Fig. 3 shows our DTMC model state transition diagram for the probabilistic load status of each PM in the data center. As depicted in Fig. 3, the model consists of three states including underload, overload, and well-utilized. So the discrete state space is defined as S = {U, O, W}. At each time step, the transition probability from one state to another is determined by the matrix P. The rows correspond to the starting state and the columns correspond to the ending state of a transition. So, p_{ij} is the transition probability from current state i to the state j at time n + 1.

$$P = \begin{bmatrix} p_{UU} & p_{UW} & p_{UO} \\ p_{WU} & p_{WW} & p_{WO} \\ p_{OU} & p_{OW} & p_{OO} \end{bmatrix} \tag{6}$$

To construct the matrix of transition probabilities the maximum likelihood estimation method is applied. Using the state definition and historical data the probability of transition can be estimated as:

$$\hat{p}_{ij} = \frac{C_{ij}}{\sum_{k \epsilon S} C_{ik}} \tag{7}$$

Where C_{ij} is the transition counts and defines the number of times state i is followed by j.

4.1.2. CTMC-Based host reliability prediction

In this paper, we have used CTMC model that presented in our previous work [11] to analysis PMs reliability. Since, exponential random variable is the only continues random variable with Markov property and hardware and software fault are commonly modelled as exponential distribution, it is assumed that the time to transit from a system state to another due to failures and recovery follows an exponential distribution. Memoryless property and relation to the

Algorithm 1: Categorization

```
Input: PMs status
Output: Categories
1:   foreach host in PM set do
2:     if PM_state = OU |OR|WU add to Critical_cat
3:       elseif PM_state = WR add to Optimal_cat
4:       elseif PM_state = UR |UU add to SubOptimal_cat
5:     end if
6:   end for
7: return categories
```

Poisson distribution are two reasons for using this distribution. Fig. 3 shows the CTMC model state transition diagram for the probabilistic reliability behavior of each PM in data center.

4.1.3. Categorization

Local agents send the obtained results from the previous step to the global agent. Based on the received information, the global agent categorizes the PMs into different categories considering the predicted status of the PMs. Algorithm 1 represents the pseudocode of the PMs categorization algorithm. The categories are comprised of six different sets, combining the Under-loaded, Overloaded and Well-utilized states obtained from DTMC, and the Reliable and Unreliable states from CTMC. Therefore, each PM will be in one of the six sets WR, OR, UR, WU, OU and UU. Unreliable state is related to semi active and fail states in CTMC model. Then these sets are divided into critical, optimal and sub optimal categories. If a PM is in the unreliable or overload status, will be included in critical category. Well-utilized and Reliable PMs are on optimal category, and the rest of the PMS are on suboptimal category. After categorization, source PMs for VM migration are specified for the next steps.

4.2. VM selection

It is necessary to migrate some VMs from the PMs in a critical state. Selecting one or more suitable VMs for migration is a crucial decision. The critical category includes OR, OU and WU sets as migration sources. To prevent unnecessary migrations and

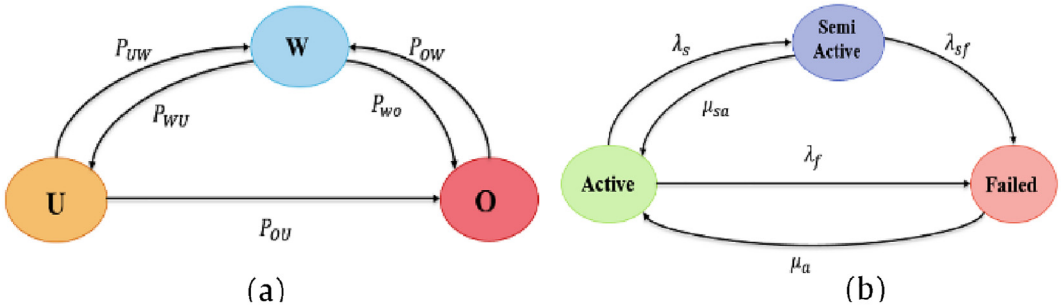

Fig. 3. State transition diagram (a) DTMC (b) CTMC.

consequently reduce the number of migrations, some constraints in VM selection have been considered in the proposed method. Compared with existing works that have used Markov chain models only for describing states and forecasting, we apply the mean first passage time concept, derived from the Markov model, during the VM selection phase. As mentioned in Section 3, we have assumed that a set of independent tasks are running on different VMs. Tasks running on each VM include sub-tasks that are executed in sequential order and use VM resources. The completion time of all the tasks on a VM is the time difference between the start and finish of a sequence of tasks on it. The estimated finishing time (EFT) of a virtual machine VM_i for all the assigned tasks can be defined as follows [33]:

$$EFT_{VM_i} = \sum_{\forall k \epsilon VM_i} ECT_{ki} \tag{8}$$

WhereECT_{ki} refers to the expected completion time of kth task on ith VM. Assuming the instruction length of the task (in million instructions) is specified, the following formula is used to calculate ECT [34]:

$$\mathrm{ECT}_{ki} = \frac{task\_length_k}{MIPS_i} \tag{9}$$

To prevent additional migrations, EFT of VMs has been used while selecting a VM. If the completion time of all the tasks on the VM is smaller than the mean first passage time obtained from CTMC or DTMC models, the VM will not be selected for migration and will continue working on the current PM. However, this condition is checked when the current status of the PM is well-utilized and reliable (WR). Therefore, the candidate VMs for migration are those that do not meet the following condition:

$$EFT_{VM_i} < MFPT_{PM_j} \tag{10}$$

If the condition is not true, the algorithm selects the VM based on two criteria. One criterion is load of the source PM after VM migration and the other is the time required for migration. The first criterion is measured based on host load changes before and after the candidate VM migration. In the case of the second criterion, the less memory required, the shorter the time required for migration. As a result, it will reduce the negative impacts of migration on the quality of service. For this purpose, the difference in CPU utilization before and after the migration is estimated to select the appropriate VM. Finally, the above mentioned criteria are combined in a criterion for scoring VMs. The values are normalized to bring them in the same direction. So, the VMs with the lowest scores are added to the migration list. The score function is defined as follows:

$$Score_i = \omega \cdot \frac{\frac{Mem_i}{BW_j} - \left(\frac{Mem}{BW}\right)_{min}}{\left(\frac{Mem}{BW}\right)_{max} - \left(\frac{Mem}{BW}\right)_{min}} + (1-\omega) \cdot \frac{\Delta U^{cpu}_{max} - \Delta U^{cpu}_{i}}{\Delta U^{cpu}_{max} - \Delta U^{cpu}_{min}} \tag{11}$$

$$\Delta U^{cpu}_{i} = U^{cpu}_{before} - U^{cpu}_{after} \tag{12}$$

Where Mem_i indicates the amount of memory occupied by VM_i, and BW_j is the available bandwidth of PM_j. U^{cpu}_{before} and U^{cpu}_{after} are CPU utilization of the PM before and after the selected VM_i migration. The higher ΔU^{cpu}_{i}, the lower the number of migrations. ΔU^{cpu}_{max} refers to the VM that makes the most change in the load of the PM after migration. The ω parameter is also considered as an adjustable weight to choose between the two criteria of minimizing the number of migrations and reducing the time of migration. After calculating the selection criterion for all VMs, they are sorted in ascending order according to their score. After each selection, it should be checked whether the PM is in a well-utilized state or not. If the current migration does not eliminate the overloading risk of PM, the next VM is selected from the list. This operation is repeated until the PM utilization reaches a normal level. The output of this phase is the migration list. Algorithm 2 shows the VM selection steps.

It is worth mentioning that in this paper, for the sake of simplicity we assume that the expected execution time of a task depends on the task length in million instructions and processing speed of VMs. But these assumptions require prior information regarding the incoming tasks and available resources that may only be known for a subset of the cloud providers. In a dynamic cloud environment, the impact of several parameters should be taken into account such as workload type and various computing resources. For example, unexpected performance variations of VMs could potentially impact the task execution time. Assuming constant or bounded VM performance also ignores the dynamic changes of the system and can lead to the pessimistic estimation of system capabilities and resource wastage. Therefore, more detailed approaches need to be applied to obtain precise estimates of task execution time. The methods which consider workflow inputs as well as the cloud features to predict the task execution time such as machine learning based approaches [35,36] or the methods that simulate the cloud fluctuating behavior [37]can be adequate solutions. By extracting the runtime parameters from workflow input data and VM type, the execution time of tasks can be predicted using a regression method or time series. Consequently, to apply the proposed VM selection method in complex environments, further investigation is needed.

4.3. ε-MOABC-based VM placement

In our proposed approach, the problem of VM placement and selecting the proper target for migrations is formulated as a multi-objective optimization problem. To solve the problem, we have used an improved version of the multi-objective bee colony algorithm called ε-MOABC [12]. The reasons for choosing this algorithm are its ability to effective management of non-dominated solutions, maintaining diversity during the evolutionary process, and achieving convergence. In contrast to many existing approaches, the main objectives of our work are energy consumption, resource wastage, and system reliability to meet SLA and QoS requirements. The main idea of integrating these three objectives is a balanced and reliable VM allocation, resulting in extra migrations avoidance in the long term. Each of these objectives is described as follows:

- Minimizing energy consumption

Reducing energy consumption is a major goal of VM consolidation. Data center power is usually determined by resources such as CPU, memory, storage, and network. However, the CPU has the most impact, and the power consumption can be described by a linear relationship of CPU utilization [14]. The following function calculates the power consumption by each PM.

$$P(u) = K \times P_{max} + (1-K) \times P_{max} \times u \tag{13}$$

Where P_{max} is the maximum power of a PM in the running state, k is the fraction of power consumed by an idle PM, and u is the CPU utilization. In this model, P_{max} is set to 250 W, as a usual value for modern servers and the value of coefficient k is set to 0.7 [12]. The total energy consumption of each PM in a period of time [t_1, t_2] can be obtained as:

$$E = \int_{t_1}^{t_2} P(u(t))dt \tag{14}$$

- Minimizing resource wastage

The next objective is to reduce the waste of data center resources. If the available resources of a PM cannot be used by any VMs, those

Algorithm 2: VM selection

```
Input: PMi (PMs in OR_list, OU_list, WU_list) // Source PMs
Output: VM migration list
Migration_list = 0
foreach PM in PMlist do
  VM_list = PM.getVmList()
    if current_state = WR
      foreach VMi in VM_list(PMj) do
        if ECT_VM < MFPT_PM
        VM_list(PMi) = VM_list(PMj) -{VMi}
        end if
      end for
    end if
    while (VM_list(PMj) = true && PMj_load > High_Tr) do
        foreach VMi in VMlist(PMj) do
          Calculate Scorei using Eq. (11)
            if Scorei < Min_Score then
              Min_Score = Scorei
              Best_VM = VMj
            end if
          end for
          Insert Best_VM in Selected_VM list
          VMlist(PMj) = VMlist-{Selected_VM}
          Calculate new PMj_load
      end while
      Migration_list = Migration_list ∪ {Selected_VM}
  end for
return Migration_list
```

resources are wasted. Therefore, this objective is considered to improve the utilize multidimensional resources. In fact, the more balanced the residual resources on a PM, the more resource capacity is utilized as much as possible. As a result, the number of active PMs and consequently energy consumption and SLA violations will decrease. The cost of resource wastage is computed as follows [38]:

$$\sum_{j=1}^{n} W_{PM_j} = \sum_{j=1}^{n} \left[y_j \times \frac{\left| \left(T_j^C - \sum_{i=1}^{m} \left(x_{ij} \cdot R_i^C \right) \right) - \left(T_j^M - \sum_{i=1}^{m} \left(x_{ij} \cdot R_i^M \right) \right) \right| + \varepsilon}{\sum_{i=1}^{m} \left(x_{ij} \cdot R_i^C \right) + \sum_{i=1}^{m} \left(x_{ij} \cdot R_i^M \right)} \right] \tag{15}$$

Where T_j^C, R_i^C, T_j^M, and R_i^M are the threshold of CPU utilization associated with each PM, the CPU demand of each VM, the threshold of memory utilization associated with each PM, and the memory demand of each VM, respectively. The binary variable y_j indicates whether PM_j is active or not and the binary variable x_{ij} indicates if VM_i is assigned to PM_j. ε is a very small positive real number and its value is set to be 0.0001.

- Maximizing system reliability

Reliability is defined as an evaluation parameter to measure the ability of the system to function correctly under specified conditions over a given time interval. Considering a single component i of a system, its reliability $R_i(t)$ are derived from its failure rate λ as follows:

$$R_i(t) = e^{-\lambda t} \tag{16}$$

Here, to calculate the PM reliability, the models presented in [9,39] are used and a combined formula is presented considering three important components of a PM including hardware, VMM, and VM. The reliability of VMs is considered as an effective factor in the consolidation ratio and subsequently in the reliability of PMs. Therefore, the reliability of a PM is defined as follows:

$$R_{PM_i}(t) = R_{HW_i}(t) R_{VMM_i}(t) R_{VM_i}(t) \tag{17}$$

Where, R_{VM_i} is the reliability of all VMs on the i*th* PM and is calculated as follows [9]:

$$R_{VM_i}(t) = \prod_{j=1}^{n} e^{-\lambda_{max_i} \times U_i \times t_j^{long}} \tag{18}$$

In this formula, λ_{max_i} is the hazard rate of PM_i at the maximum utilization, U_i is the PM utilization, and t_j^{long} is the longest task running on the VM_j. Suppose to PM failures are independent, the overall system reliability is defined as:

$$R_{sys}(t) = \prod_{i=1}^{m} R_{PM_i}(t) \tag{19}$$

Therefore, according the above mentioned definitions, the optimization problem to find the optimum mapping between VMs and PMs can be formulated as follows:

$$Minimize : E_{total} = \sum_{j=1}^{n} E_{PM_j} \tag{20}$$

$$Resource\_Westage = \sum_{j=1}^{n} W_{PM_j} \tag{21}$$

$$Maximize : R_{sys}(t) = \prod_{j=1}^{n} R_{PM_j}(t) \tag{22}$$

Subject to

$$\sum_{j=1}^{n} x_{ij} = 1, x_{ij} = 0 or 1, i = [1, 2, \cdots, m] \tag{23}$$

$$\sum_{i=1}^{m} l_i^{CPU} x_{ij} < C_j^{CPU} \cap \sum_{i=1}^{m} l_i^{mem} x_{ij} < C_j^{mem} \cap \sum_{i=1}^{m} l_i^{bw} x_{ij} < C_j^{bw} \tag{24}$$

Constraint presented in Eq. (23) implies that each VM can be assigned to only one PM. Constraint described in Eq. (24) ensures that the allocated resources do not exceed the total available resource in that particular PM.

4.3.1. ε-MOABC Algorithm

The ABC algorithm is an evolutionary algorithm that is inspired by the foraging behavior of honey bees [40]. The MOABC is a multi-objective variant of the ABC algorithm and uses the concept of Pareto dominance to determine the flight direction of a bee [41,42]. Similar to the standard ABC, MOABC also involves three types of bees that perform different tasks by collaborating with one another. The ε-MOABC is an improved version of the MOABC algorithm which uses the quality indicator to solve multi-objective problems [12]. In this approach, the food sources with higher quality are selected according to the binary indicator I_{ε^+}, and non-dominated solutions are maintained in an external archive. In fact, binary quality indicator is adopted to calculate the Pareto optimal set in the feasible region and to balance convergence and diversity well. A quality indicator is a function that maps k Pareto set approximations to a real number. The binary additive ε-indicator

(I_{ε^+}) gives the minimum distance by which a Pareto set approximation needs to or can be translated in each dimension in the objective space such that another approximation is weakly dominated. The concept formally defined as [43]:

$$I_{\varepsilon^+}(A,B) = min_{\varepsilon}\{\forall x_2 \epsilon B \exists x_1 \epsilon A : f_i(x_1) - \varepsilon \leq f_i(x_2) for i \epsilon \{1,2,\cdots,n\}\} \quad (25)$$

In ε-MOABC algorithm, this indicator is applied to update both of the individuals in the population and the external archive. Indicator based fitness assignment tries to rank population members based on their eligibility concerning to the goal of optimization. Another difference in ε-MOABC is the mutation operator that is derived from the mutation operator of the Differential Evolution (DE) algorithm to escape the local optima.

4.3.2. VMs to PMs mapping

In this paper, we have used the ε-MOABC algorithm to solve the VM placement as a multi-objective optimization problem. The proposed algorithm starts with the input parameters including the list of migrated VMs, destination PMs, number of solutions and maximum number of iterations. The algorithm steps are as follows:

Step 1: initialization

In this step, food sources (solutions) are randomly generated. Each food source is initialized according to the following equation:

$$x_{id} = round(lb_d + rand(0.1) * (ub_d - lb_d)) \quad (26)$$

Where $i = 1.2.\cdots.Foodnumber$ is the index of the food source, and d specifies the dimension of each food source. The rand () function produces a random number distributed uniformly over the interval [0, 1]. $ub_d$and lb_dindicate the upper and lower bounds of the dth dimension of a food source, respectively. Finally, a variable called $trial_i$ will be assigned to each food source i whose value is set to zero. If a food source could not get optimized in a number of trials, then the food source is abandoned, and its related employed bee turns into a scout bee to find a new food source randomly.

In our optimization problem, the dimension of each food source is equal to the number of VMs in the migration list. Each food source is represented as:

$$x_i = \left[x_{i1}, x_{i2}, \cdots, x_{iN_{VM}}\right], i = 1,2,\cdots,Foodnumber$$

The value of $x_{i.m}$ is determined using Eq. (25), $ub_d = 1$, and $lb_d = N_{PM}$. For instance to generate x_i, if we assume that there are 10 VMs in migration list and 6 PMs as feasible targets, then the solution x_i can be represented as follow:

$$x_i = [4612254361]$$

According to this vector, the first VM should be placed at the fourth PM, the second VM at the sixth PM and so on.

Step 2: Sending employed bees

Each employed bee i explores a food source, and then perform a local random search to calculate a new temporary position v_{id}around it. The number of employed bees is equal to the number of food sources. To escape the local optima trap, the differential evolution mutation is applied and calculated according to following formula:

$$v_{id} = x_{r_1 d} + F \times (x_{r_2 d} - x_{r_3 d}) \quad (27)$$

Where r_1, r_2, r_3 are the indexes of three random solutions which are different from i. the value of d is also selected randomly, and $F \epsilon [-1,1]$ is a real number that adjusts the impact to the difference vector. Afterward, the fitness of the new solution is evaluated. If a better value is obtained, then the current food source is replaced by the food source in a new position, and the trail value for this new food source is set to zero. Otherwise, the value of the trial for current food source will be incremented by one. The fitness of every food source is evaluated using binary indicator I_{ε^+} , which is calculated according to the following formulas[43,44]:

$$I_{\varepsilon^+}(x_1,x_2) = max_{i \epsilon \{1,\cdots,n\}}(f_i(x_1) - f_i(x_2)) \quad (28)$$

$$Fit(x) = I(P\{\{x_1\},x_1) = \sum_{x_2 \epsilon P\{\{x_1\}} -e^{-I_{\varepsilon^+}(x_2,x_1)/\kappa} \quad (29)$$

Where x_1, x_2 are the food sources, and n is the number of objectives. The $f_i(x_1)$ is the triple objective vector function in our problem. $I(P\{\{x_1\},x_1)$ is the fitness of an individual x_1 and computes the overall quality of each solution with respect to the rest of the population.$\kappa > 0$ represents the scaling factor that is set to 0.05 in this paper.

Step 3: Sending onlooker bees

After the optimization of food sources was accomplished by employed bees, they come back to the hive to share their information with the onlooker bees. Each of the onlooker bees chooses one of the food sources based on their fitness. The higher the fitness value, the more likely the higher quality food sources will be around them. In theε-MOABC algorithm, in each generation of the evolution process, food resources are ranked in descending order based on their fitness value computed by I_{ε^+} indicator. The probability of choosing the proper food source is calculated according to the sorted number and scale-invariance power low. Indeed, the food source with the appropriate fitness is exploited based on the following formula [12]:

$$P(s) = \frac{s^{-\beta}}{1^{-\beta} + 2^{-\beta} + \cdots + s^{-\beta} + \cdots + t^{-\beta}}, 1 \leq s \leq t, \beta \geq 0 \quad (30)$$

Where β is an adjustable parameter and s indicates the rank of fitness of the food sources. The worst rank is t (equal to the number of food sources), and the best rank is 1. The mutation and fitness evaluation methods for onlooker bees are the same as those of the employed bees.

Step 4: Sending scout bees

In this step, the algorithm will find the abandoned food sources that could not be optimized in the limited cycles, and their trial limits are exceeded. Then the associated employed bee turns into a scout bee, and the new food source is generated in the same manner as the initialization step. In other word, a scout bee resets each food source whose migration plan to VMs placement has not been updated for several iterations.

Step 5: Updating archive

A fixed size external archive is used to store the non-dominated solutions found during the search process. If the number of non-dominated solutions increases and exceeds the maximum archive capacity, some solution with the smallest fitness are removed one by one. After deleting every individual x^*, the fitness of the remaining solutions has to be updated as follows:

$$Fit(x) = Fit(x) - I(P\{\{x^*\},x^*) \quad (31)$$

This removal method causes the last Pareto front to be distributed unevenly. In fact, indicator based evolutionary algorithms do not require any additional diversity preservation mechanism [43].

Step 6: Selecting the best solution

After obtaining the Pareto-optimal set, we need to decide on the best solution, which here is the VM to PM mapping in the form of a migration plan. For this purpose, the optimal solution is selected based on the minimum Euclidean distance to the origin among all the non-dominated solutions in the archive, the solutions with the highest binary indicator value are near to optimal.

Algorithm 3: ε –MOABC based VM placement

Input: Migration_list, WR_list, UR_list
Output: Migration plan(Mapping of each VMs to PMs)
1: Set the parameter Population size, Archive size, trial_limit, $iter_{max}$.
2: Randomly initialise a population $X = (x_1,x_2,\ldots,x_N)$
3: Evaluate fitness functions f_1 , f_2 and f_3 for any individual x_i
Start the optimization process:
4: While (iter < $iter_{max}$)
5: for i = 1 to Foodnumber //Employed Bees
6: Select one dimension d and 3 neghibors p, k and q randomly
7: Calculate new solution using Eq. (27)
8: Evaluate fitness functions for new solution(obtaining new position)
9: Calculate fitness based on indicator $\boldsymbol{I}_{\varepsilon^+}$ using Eq. (28) and Eq.(29)
10: if ($I(v_i) > I(x_i)$)
11: Add v_i to the archive
12: $x_i = v_i$
13: $trial_i = 0$
14: else
15: $trial_i$ ++;
16: end if
17: end for
18: Calculate the probabilities of each solution using Eq. (30)// OnlookerBees
19: i = 0 , t = 0
20: while (t < FoodNumber)
21: if (rand < P_i)
22: t = t + 1
23: Select one dimension d and 3 neghibors p, k and q randomly
24: Evaluate fitness functions for new solution
25: Calculate fitness based on indicator $\boldsymbol{I}_{\varepsilon^+}$ using Eq. (28) and Eq.(29)
26: if ($I(v_i) > I(x_i)$)
27: Add v_i to the archive
28: $x_i = v_i$
29: $trial_i = 0$
30: else
31: $trial_i$ ++;
32: end if
33: i = i + 1;
34: if (i== FoodNumber)
35: i = 1
36: end If
37: end While
38: for i = 1 to FoodNumber // Scout Bees
39: if ($max(trial_i) >$ trial_limit)
40: for d = 1 to D
41: $\boldsymbol{x_{id}} = \boldsymbol{round}(\boldsymbol{lb_d} + \boldsymbol{rand}(0.1) * (\boldsymbol{ub_d} - \boldsymbol{lb_d}))$
42: end For
43: $trial_i = 0$;
44: end If
45: end For
46: Update Archive()
47: end while
48: Select the best solution from archive
49:Return Migration plan

Table 1
The virtual machine instances.

VM type	CPU frequency (MIPS)	RAM (GB)
High-CPU medium instance	2500	0.85
Extra-large instance	2000	3.75
Small instance	1000	1.7
Micro instance	500	0.613

5. Experiments and Discussion

In this section, the performance of proposed task VM consolidation approach is evaluated according to the obtained results. Experiment setup, workload model, evaluation metrics, and experimental results are presented as follows.

5.1. Experimental Setup

Implementing the VM consolidation algorithms on a real infrastructure for large scale experiments is very difficult. so in this study we have employed CloudSim toolkit [45] as the simulation platform. The experiments simulate a data center comprised of 800 heterogeneous PMs, half of which are HP ProLiant ML110 G4 (Intel Xeon 3040 2 Cores 1860 MHz, 4 GB) servers, and the other half are HP ProLiant ML110 G5 (Intel Xeon 3075 2 Cores 2260 MHz, 4 GB). VMs are supposed to correspond to Amazon EC2 instance types with the only exception that all the VMs are single-core, because of the fact that the workload data used for the simulations come from single-core VMs. There are four types of VMs in the experiments as shown in Table 1. After each round of VMs consolidation, VMs resource demands changes according to workload data. The HighTR and LowTR thresholds have assumed equal to 0.8 and 0.4, respectively. The archive size is 30, and the population size and the maximum number of iterations are 50 that are set based on the experience values. We performed each experiment 10 times for our proposed approach, and the mean values for different metrics are reported.

5.2. Workload model

In order to make the results of simulation more realistic, it is important to conduct experiments using workload traces from a real system. We have used data that provided as a part of the CoMon project, a monitoring infrastructure for Planet Lab [46]. In this project, the data on the CPU utilization is obtained every five minutes by more than a thousand virtual machines from servers located at more than 500 places around the world., randomly. Table 2 shows the characteristics of 10 different days that have chosen from the workload traces randomly.

5.3. Performance evaluation metrics

Energy consumption and SLA violations are two important factors in VM consolidation problem. In order to reasonably evaluate the efficiency of our proposed approach, we adopt several metrics that were presented by beloglazov et al. [30]. These metrics are included: service level agreement violation time per active host (SLATAH), overall performance degradation caused by migrations (PDM), SLA violations (SLAV), energy consumption (EC), VM migrations, and energy and SLA violations (ESV).

QoS requirements are commonly formalized in the form of SLAs, which can be determined in terms of such characteristics as

Table 2
The properties of Planetlab trace (CPU utilization).

Date	Number of VMs	Mean(%)	St. dev. (%)
03/03/2011	1052	12.31	6.68
06/03/2011	898	11.44	6.77
09/03/2011	1061	10.70	7.35
22/03/2011	1516	9.26	6.24
25/03/2011	1078	10.56	6.32
03/04/2011	1463	12.39	7.03
09/04/2011	1358	11.12	6.95
11/04/2011	1233	11.56	7.13
12/04/2011	1054	11.54	7.22
20/04/2011	1033	10.43	8.10

minimum throughput or maximum response time delivered by the deployed system [30]. There are two basic metrics to depict an SLA violation: SLATAH and PDM. The SLATAH is defined as Eq. (32), measures the percentage of time during which active hosts have experienced CPU utilization of 100%.

$$SLATAH = \frac{1}{n}\sum_{i=1}^{n}\frac{T_i^s}{T_i^a} \tag{32}$$

Where n is the total number of physical machines, T_i^s is the total time of SLAV caused by the CPU resource overload of PM_i, T_i^a is the running time of PM_i. Another metric, PDM is calculated as follow:

$$PDM = \frac{1}{m}\sum_{j=1}^{m}\frac{C_j^d}{C_j^r} \tag{33}$$

Where m is the number of VMs, C_j^d is the unsatisfied CPU required capacity caused by the migration of vm_j, and C_j^r is the CPU capacity requested by vm_j. SLAV is a combined metric of two aforementioned metrics that evaluates a single-day QoS of the data center:

$$SLAV = SLATAH \times PDM \tag{34}$$

ESV as described in Eq. (35), is a combined metric consist of energy consumption of a data center per day and the level of SLA violations. Energy consumption and SLA violations are typically negatively correlated as energy can usually be decreased by the cost of the increased level of SLA violations [30]. A lower estimation of ESV indicates that energy saving is higher than the SLA violations.

$$ESV = EC \times SLAV \tag{35}$$

5.4. Experimental results and analysis

In this section, To investigate the impact of our method on the performance metrics mentioned in section 5.3, we compare the proposed consolidation approach with some traditional methods including LR-MMT, LR-MC, LR-RS, MAD-MMT, MAD-MC, MAD-RS, IQR-MMT, IQR-MC, and IQR-RS [12]. LR method predicted the CPU utilizations using local regression. The MAD and IQR measured the workload stability by calculating median absolute deviation and interquartile range of CPU utilization. MMT as a VM selection policy preferred the VMs with a minimum migration time to migrate. MC policy selects those VMs that have the highest correlation of the CPU utilization with other VMs, and the Random Selection (RS) policy selects VMs according to a uniformly distributed discrete random variable. These methods apply the PABFD [18] algorithm in the VM placement phase. The safety parameter is set to 1.2 in experiments. The CTMC model parameter default values are found in previous work [11].

Table 3 shows the comparison between our proposed MOABC-VMC approach and the other methods in terms of different metrics. The MOABC-VMC results include the mean and standard deviation for 10 runs. The obtained results indicate that energy consumption is reduced by MOABC-VMC algorithm compared to other methods. The next metric in the Table is SLAV, which demonstrates the ability of the algorithms to guarantee the quality of service. The experimental results indicate that the MOABC-VMC and MAD-RS methods have the lowest and highest value of SLAV, respectively. RE-VMC and LR-MMT methods also have optimal values. However, the results show that SLAV of MOABC-VMC is only 18% of LR-MMT's SLAV. Therefore, the proposed approach is more successful in guaranteeing QoS.

As can be seen in Table 3, the next index is VM migrations and the methods are compared in terms of the number of VM migration based on the experimental results. Additional migrations increase energy consumption, hence the number of migrations affects energy costs and the QoS. MOABC-VMC has the lowest number of VM migrations compared to the other methods. In fact, selecting the reliable PMs as the destination of migration, and the conditional selection of migrated VMs using multiple criteria, on the other hand, results in a significant reduction in the number of migrations. This reduction will improve the quality of service. The next metric is ESV, which is a comprehensive metric in assessing energy consumption, number of migrations, and quality of service. The lower the metric, the better the algorithm's performance in energy saving and guaranteeing the quality of service. Experimental results indicate that the comprehensive performance of MOABC-VMC is considerably higher than the others. The MAD-RS method has the worst ESV, which is 20.4 times larger than our proposed approach. Therefore, according to the results, it can be concluded that the MOABC-VMC method has been achieved to the optimization objectives. The standard deviation shows that the variability of the results is negligible between the different runs.

We investigate the energy consumption improvement of our proposed algorithm rather than other algorithms, as shown in Fig. 4. It is obvious that MOABC-VMC consumes the least amount of energy, and reduces energy consumption by 11.35% compared to RE-VMC. This indicates that the accuracy of PM load estimation by the DTMC model is acceptable in the proposed method and has a positive effect on VMs reassignment and the energy consumption improvement. It is also can be realized that the Methods with LR policy to determine overload PMs have lower power consumption than the two IQR and MAD policies. The RE-VMC also uses the LR method to estimate the status of PMs in terms of workload. Fig. 5 shows the simulation results in terms of the SLATAH metric under different VM consolidation methods. MOABC-VMC method guarantees the QoS of active PMs and reduces the PMs overload risk. The reason for this improvement is taking the current status along with the predicted status into account, which effectively leads to proper destination PM selection. The next metric is PDM, which also affects SLA violations.

Fig. 6 shows that the MOABC-VMC method effectively reduces the probability of QoS degradation due to service interruption during migration. This is because the proposed method has attempted to prevent unnecessary VM migrations. As mentioned before, in the proposed VM selection method, only those VMs that cannot complete their tasks in the current PM are selected for migration. This is very important in maintaining the quality of service. On the other hand, in the proposed method, the load and reliability of PMs are considered simultaneously. So, the selection of less risky PMs as the migration destination results in preventing additional migrations and consequently reducing the total number of migrations. This reduction significantly effects on the PDM metric.

Fig. 7 shows the overall number of migrations that is one of the most important evaluation metrics in VM consolidation methods.

Table 3
Simulation results of different algorithms.

Method	EC (KWh)	SLAV	VM Migrations	ESV (%)
LR-MMT	162.53	0.48244	32,871	78.4110
LR-MC	149.25	0.74538	27,095	111.2480
LR-RS	147.88	0.76694	26,540	113.4151
IQR-MMT	189.13	0.58384	35,944	110.4217
IQR-MC	182.75	0.90191	32,241	164.8241
IQR-RS	179.38	0.99358	30,583	178.2284
MAD-MMT	186.44	0.62726	34,722	116.9464
MAD-MC	179.15	1.01375	29,009	181.6133
MAD-RS	177.65	1.04739	28,814	186.0688
RE-VMC	118.71	0.13728	9122	16.2965
MOABC-VMC Mean (St.dev.)	105.24(1.264)	0.08635(0.017)	6717(24.093)	9.0875(1.044)

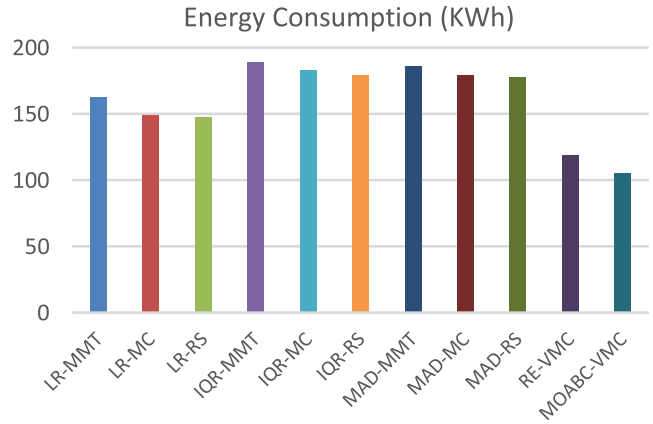

Fig. 4. The energy consumption of algorithms.

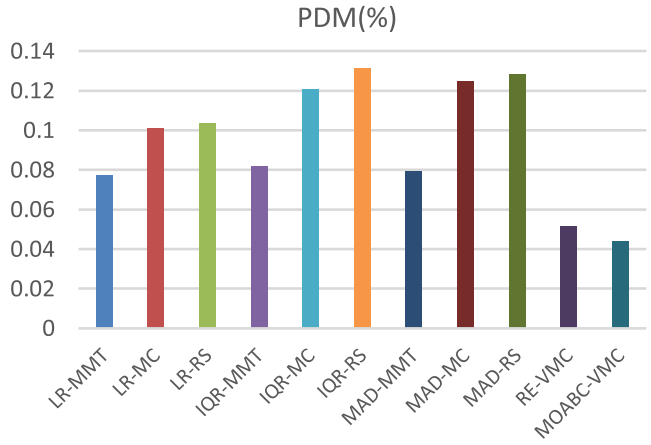

Fig. 6. Comparison of PDM.

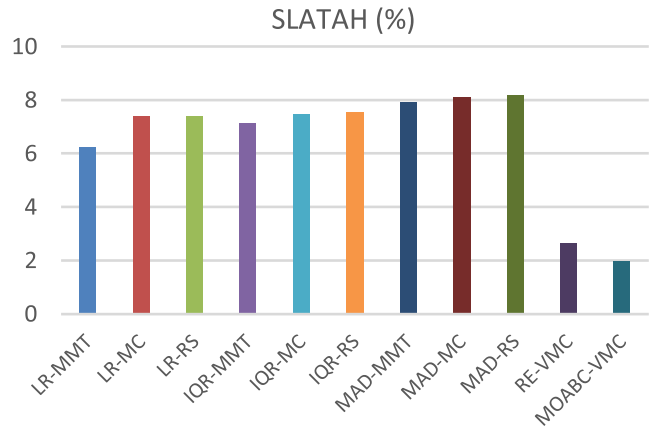

Fig. 5. Comparison of SLATAH.

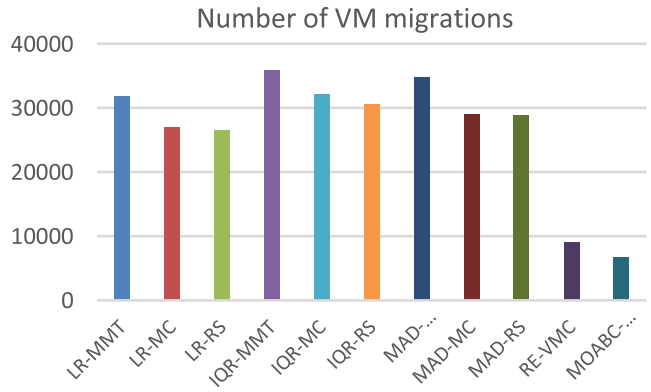

Fig. 7. Comparison of the number of VM migrations.

As can be seen, the number of migrations triggered by MOABC-VMC is fewer than the other methods. This observation can be described by the fact that the proposed method simultaneously considers several factors in VM migration. Considering the reliability while selecting the target PMs leads to prevent re-migrations of VMs. Furthermore, in the MOABC-VMC method, the load status of PMs is predicted by the DTMC Markov model, which reduces the probability of migration to PMs whose state will be overload in the future. To reduce the number of migrations, in the VM selection phase, the time to complete the running tasks on the VMs is also considered. Therefore, the VM that is capable to complete its tasks before the PM enters the critical state will not select to migration. Fig. 8 shows the variation of the number of migrations in consecutive cycles of the VM consolidation process. In our simulation, the total amount of cycles is 288 per day. Indeed, a cycle of VM consolidation is conducted every 5 min on each active host. As depicted in the figure, the number of migrations within the preliminary cycles is greater than other algorithms. One reason is that the proposed approach selects unreliable PMs in addition to overloaded PMs as VM migration source, which leads to the number of migrations increases in the initial cycles. But it is obvious that as time passes, by identifying appropriate PMs, VMs are placed on more reliable PMs with normal CPU utilization. So, extra VM migrations are avoided and the number of VM migrations is significantly degraded. Another reason is that in the proposed approach, all VMs on underloaded PMs are placed in the migration list from the beginning and reallocate to more adequate PMs. As can be seen in the figure, except for the primary cycles in the rest of the cycles, the number of migrations is less than 30, which is significantly lower than the other methods. Decreasing the number of

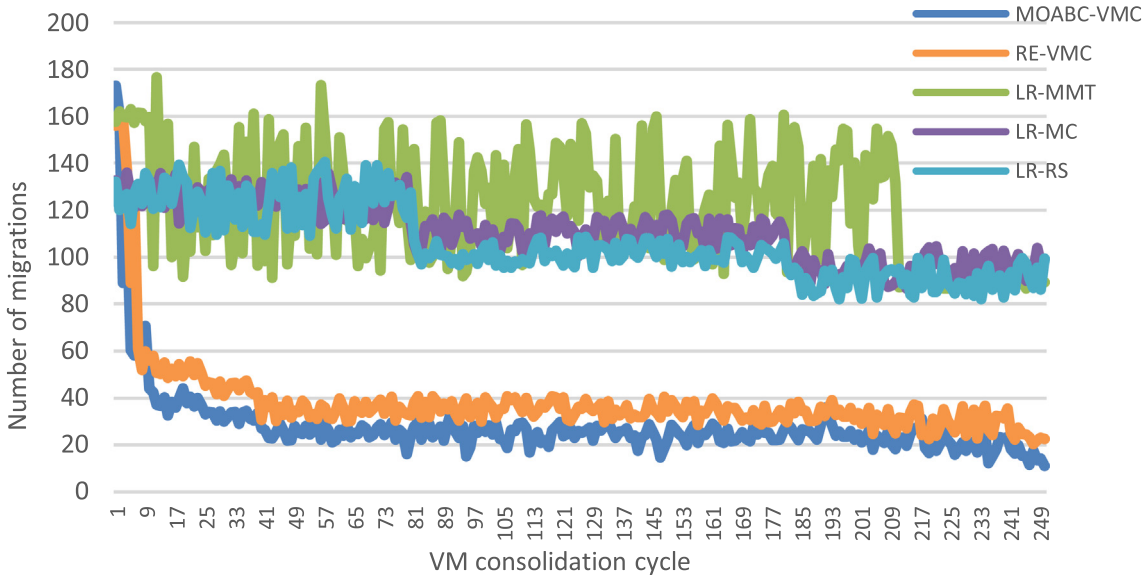

Fig. 8. Variation of the number of VM migrations.

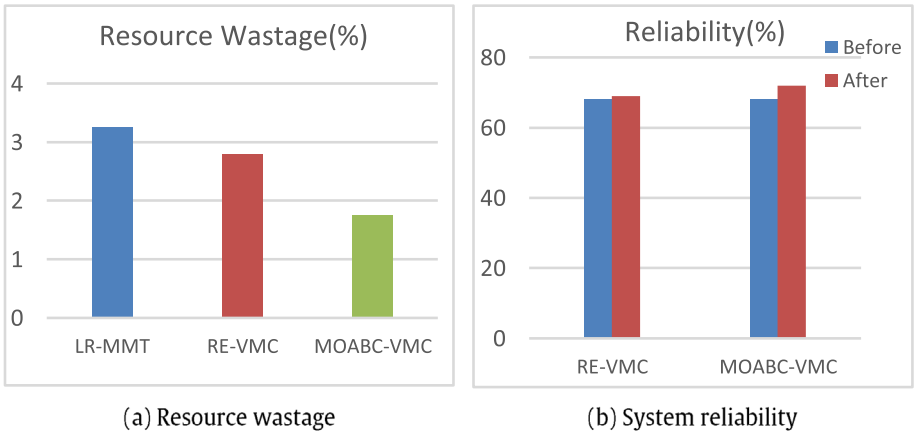

Fig. 9. Comparison of the resource wastage and variation of the system reliability.

migrations in subsequent consolidation cycles proves the efficiency of the MOABC-VMC method.

Fig. 9 shows the amount of resource wastage and the variation of the system reliability as two important VM placement objectives. Since all traditional methods in our previous experiments use the PABFD algorithm in the VM placement phase, only one of them is used to the comparison in Fig. 9(a). We can clearly observe that the proposed method outperforms two other methods and has the minimum resource wastage. The obtained result indicates that the resources of the running PMs like CPU and memory, are properly utilized. Consequently, a large number of VMs can be accommodated in a less number of PMs. In Fig. 9(b), the variation of the system reliability before and after the VM consolidation. The proposed method maintains the system reliability to an acceptable level that is important to prevent the additional migrations as well as the risk of system failures due to the greedy consolidation of VMs.

In general, according to the simulation result, the optimization objectives are realized by MOABC-VMC, with sufficient consideration of several factors in a real data center.

6. Conclusion

Energy consumption is one of the important issues in cloud data ceners. Dynamic VM consolidation methods enable data centers to reduce the consumed energy by minimizing the number of active PMs. In this work, we studied the dynamic VM consolidation problem and presented a novel multi-objective optimization solution to efficiently achieve a set of non-dominated solutions that minimizes energy consumption and resource wastage, and maximizes the system reliability, simultaneously. In particular, we have introduced a discrete-time Markov chain model to PMs status prediction and then used it along with the CTMC model proposed in our previous work. These models make it possible for a more efficient PMs categorization and as a consequence more accurate decision making to select the migration source and destination PMs. So, additional migrations due to PMs overloading or unreliability are reduced which leads to improve energy consumption and decrease the SLA violations. During the VM selection phase, it has also been attempted to reduce the unnecessary migrations as far as possible in light of the anticipated status of the PMs. Afterward, finding the

optimal VM to PM mapping problem is solved as a multi-objective problem using ε-MOABC algorithm. We have evaluated our proposed approach and the simulation results demonstrated that our proposed solution outperforms other methods in terms of defined objectives and metrics. As future work directions, we seek to evaluate the performance of the proposed algorithm with more workloads of real datacenters. Towards this end, task scheduling and VM consolidation problems need to be integrated as one problem in a holistic manner considering effective parameters on tasks execution time in a dynamic cloud environment. We also plan to conduct further studies to address more objectives such as resource overcommitment and bandwidth resource constraints.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] P. Mell and T. Grance, "The NIST definition of cloud computing," 2011.
[2] R.W. Ahmad, A. Gani, S.H.A. Hamid, M. Shiraz, A. Yousafzai, F. Xia, A survey on virtual machine migration and server consolidation frameworks for cloud data centers, J. Network Comput. Appl. 52 (2015) 11–25.
[3] A. Varasteh, M. Goudarzi, Server consolidation techniques in virtualized data centers: a survey, IEEE Syst. J. 11 (2) (2017) 772–783.
[4] J.E. Smith, R. Nair, The architecture of virtual machines, Computer 38 (5) (2005) 32–38.
[5] C. Clark et al., "Live migration of virtual machines," in Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2, 2005, pp. 273-286.
[6] W. Deng, F. Liu, H. Jin, X. Liao, H. Liu, Reliability-aware server consolidation for balancing energy-lifetime tradeoff in virtualized cloud datacenters, Int. J. Commun. Syst. 27 (4) (2014) 623–642.
[7] A. Varasteh, F. Tashtarian, and M. Goudarzi, "On Reliability-Aware Server Consolidation in Cloud Datacenters," arXiv preprint arXiv:1709.00411, 2017.
[8] Y. Sharma, B. Javadi, W. Si, D. Sun, Reliability and energy efficiency in cloud computing systems: Survey and taxonomy, J. Network Comput. Appl. 74 (2016) 66–85.
[9] Y. Sharma, W. Si, D. Sun, B. Javadi, Failure-aware energy-efficient VM consolidation in cloud computing systems, Future Gener. Comput. Syst. 94 (2019) 620–633.
[10] A. Greenberg, J. Hamilton, D. A. Maltz, and P. Patel, "The cost of a cloud: research problems in data center networks," ed: ACM New York, NY, USA, 2008.
[11] M.H. Sayadnavard, A.T. Haghighat, A.M. Rahmani, A reliable energy-aware approach for dynamic virtual machine consolidation in cloud data centers, J. Supercomput. 75 (4) (2019) 2126–2147.
[12] J. Luo, Q. Liu, Y. Yang, X. Li, M.-R. Chen, W. Cao, An artificial bee colony algorithm for multi-objective optimisation, Appl. Soft Comput. 50 (2017) 235–251.
[13] A. Beloglazov, "Energy-efficient management of virtual machines in data centers for cloud computing," 2013.
[14] A. Beloglazov, J. Abawajy, R. Buyya, Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing, Future Gener. Comput. Syst. 28 (5) (2012) 755–768.
[15] A. Beloglazov and R. Buyya, "Energy efficient resource management in virtualized cloud data centers," in: Proceedings of the 2010 10th IEEE/ACM international conference on cluster, cloud and grid computing, 2010, pp. 826-831: IEEE Computer Society.
[16] L. Grit, D. Irwin, A. Yumerefendi, and J. Chase, "Virtual machine hosting for networked clusters: Building the foundations for autonomic orchestration," in: Proceedings of the 2nd International Workshop on Virtualization Technology in Distributed Computing, 2006, p. 7: IEEE Computer Society.
[17] B. Speitkamp, M. Bichler, A mathematical programming approach for server consolidation problems in virtualized data centers, IEEE Trans. Serv. Comput. 3 (4) (2010) 266–278.
[18] A. Beloglazov, R. Buyya, Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers, Concurrency and Comput.: Practice Experience 24 (13) (2012) 1397–1420.
[19] S. Zhang, Z. Qian, Z. Luo, J. Wu, S. Lu, Burstiness-aware resource reservation for server consolidation in computing clouds, IEEE Trans. Parallel Distrib. Syst. 27 (4) (2016) 964–977.
[20] H. Shen, L. Chen, Compvm: a complementary vm allocation mechanism for cloud systems, IEEE/ACM Trans. Networking (TON) 26 (3) (2018) 1348–1361.
[21] A. Beloglazov, R. Buyya, Managing overloaded hosts for dynamic consolidation of virtual machines in cloud data centers under quality of service constraints, IEEE Trans. Parallel Distrib. Syst. 24 (7) (2013) 1366–1379.
[22] A. Ponraj, Optimistic virtual machine placement in cloud data centers using queuing approach, Future Gener. Comput. Syst. 93 (2019) 338–344.
[23] Z. Li, C. Yan, X. Yu, N. Yu, Bayesian network-based virtual machines consolidation method, Future Gener. Comput. Syst. 69 (2017) 75–87.
[24] T. Mahdhi, H. Mezni, A prediction-Based VM consolidation approach in IaaS Cloud Data Centers, J. Syst. Softw. 146 (2018) 263–285.
[25] A. Ashraf, I. Porres, Multi-objective dynamic virtual machine consolidation in the cloud using ant colony system, Int. J. Parallel Emergent Distrib. Syst. 33 (1) (2018) 103–120.
[26] F. Farahnakian, A. Ashraf, T. Pahikkala, P. Liljeberg, J. Plosila, I. Porres, H. Tenhunen, Using ant colony system to consolidate VMs for green cloud computing, IEEE Trans. Serv. Comput. 8 (2) (2015) 187–198.
[27] M. Terra-Neves, I. Lynce, V. Manquinho, Virtual machine consolidation using constraint-based multi-objective optimization, J. Heuristics 25 (3) (2019) 339–375.
[28] H. Zhao, J. Wang, F. Liu, Q. Wang, W. Zhang, Q. Zheng, Power-aware and performance-guaranteed virtual machine placement in the cloud, IEEE Trans. Parallel Distrib. Syst. 29 (6) (2018) 1385–1400.
[29] H. Mi, H. Wang, G. Yin, Y. Zhou, D. Shi, and L. Yuan, "Online self-reconfiguration with performance guarantee for energy-efficient large-scale cloud computing data centers," in: Services Computing (SCC), 2010 IEEE International Conference on, 2010, pp. 514-521: IEEE.
[30] Z. Li, C. Yan, L. Yu, X. Yu, Energy-aware and multi-resource overload probability constraint-based virtual machine dynamic consolidation method, Future Gener. Comput. Syst. 80 (2018) 139–156.
[31] Z. Li, X. Yu, L. Yu, S. Guo, V. Chang, Energy-efficient and quality-aware VM consolidation method, Future Gener. Comput. Syst. 102 (2020) 789–809.
[32] J. K. Ghosh, "Introduction to Modeling and Analysis of Stochastic Systems, by VG Kulkarni," International Statistical Review, 80(3), pp. 487-487, 2012.
[33] J.-T. Tsai, J.-C. Fang, J.-H. Chou, Optimized task scheduling and resource allocation on cloud computing environment using improved differential evolution algorithm, Comput. Oper. Res. 40 (12) (2013) 3045–3055.
[34] M.A. Elaziz, S. Xiong, K.P.N. Jayasena, L. Li, Task scheduling in cloud computing based on hybrid moth search algorithm and differential evolution, Knowl.-Based Syst. 169 (2019) 39–52.
[35] T.-P. Pham, J.J. Durillo, T. Fahringer, Predicting workflow task execution time in the cloud using a two-stage machine learning approach, IEEE Trans. Cloud Comput. 8 (1) (2020) 256–268.
[36] M. H. Hilman, M. A. Rodriguez, and R. Buyya, "Task runtime prediction in scientific workflows using an online incremental learning approach," in: 2018 IEEE/ACM 11th International Conference on Utility and Cloud Computing (UCC), 2018, pp. 93-102: IEEE.
[37] R. Mathá, S. Ristov, T. Fahringer, R. Prodan, Simplified workflow simulation on clouds based on computation and communication noisiness, IEEE Trans. Parallel Distrib. Syst. 31 (7) (2020) 1559–1574.
[38] Y. Gao, H. Guan, Z. Qi, Y. Hou, L. Liu, A multi-objective ant colony system algorithm for virtual machine placement in cloud computing, J. Comput. Syst. Sci. 79 (8) (2013) 1230–1242.
[39] B. Wei, C. Lin, and X. Kong, "Dependability modeling and analysis for the virtual data center of cloud computing," in: High Performance Computing and Communications (HPCC), 2011 IEEE 13th International Conference on, 2011, pp. 784-789: IEEE.
[40] D. Karaboga, "An idea based on honey bee swarm for numerical optimization," Technical report-tr06, Erciyes university, engineering faculty, computer, 2005.
[41] R. Akbari, R. Hedayatzadeh, K. Ziarati, B. Hassanizadeh, A multi-objective artificial bee colony algorithm, Swarm Evol. Comput. 2 (2012) 39–52.
[42] W. Zou, Y. Zhu, H. Chen, H. Shen, A novel multi-objective optimization algorithm based on artificial bee colony, in: Proceedings of the 13th annual conference companion on Genetic and evolutionary computation, 2011, pp. 103–104.
[43] E. Zitzler and S. Künzli, "Indicator-based selection in multiobjective search," in: International conference on parallel problem solving from nature, 2004, pp. 832-842: Springer.
[44] M. Basseur, A. Liefooghe, K. Le, E.K. Burke, The efficiency of indicator-based local search for multi-objective combinatorial optimisation problems, J. Heuristics 18 (2) (2012) 263–296.
[45] R.N. Calheiros, R. Ranjan, A. Beloglazov, C.A.F. De Rose, R. Buyya, CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms, Software: Practice Experience 41 (1) (2011) 23–50.
[46] K. Park, V.S. Pai, CoMon: a mostly-scalable monitoring system for PlanetLab, ACM SIGOPS Operat. Syst. Rev. 40 (1) (2006) 65–74.